Universidad Católica de Oriente Facultad de Ingeniería	**Compiladores – Entrega Final** Semestre 2-2026

Enunciado

Esta entrega constituye el **producto final del proyecto del curso**. En ella, el estudiante debe integrar todas las fases del compilador desarrolladas a lo largo del semestre (análisis léxico, sintáctico y semántico) y agregar la fase de **traducción orientada por la sintaxis (SDT)**, de modo que el compilador sea capaz de recibir una cadena en el lenguaje fuente definido y producir, como salida, un texto válido en el lenguaje de destino elegido desde el inicio del proyecto.

> **Recuerde:** las fases léxica, sintáctica y semántica ya fueron calificadas en entregas anteriores. Esta entrega evalúa la correcta **integración completa** y, especialmente, la implementación de la fase SDT. Se espera que el trabajo previo sea incorporado sin regresiones: un compilador que haya funcionado correctamente en entregas anteriores debe seguir haciéndolo en esta entrega.

Parte 1 – Gramática ampliada completa

Presente la gramática ampliada final de su proyecto. Esta gramática debe integrar los tres tipos de anotaciones ya conocidos y usar la **convención de colores** establecida en entregas anteriores:

Color	Contenido
Rojo	Reglas originales de la gramática (producción sintáctica sin ampliar)
Fondo ámbar	Reglas semánticas (atributos, restricciones y acciones de validación)
Azul	Reglas de traducción orientadas por la sintaxis – acciones SDT (generación de código en el lenguaje destino)

La gramática ampliada final debe incluir:

- Las **reglas originales de la gramática** (texto rojo), idénticas a las presentadas en entregas anteriores.
- Las **reglas semánticas** ya definidas en el Quiz 4, con sus atributos, dominio de valores y acciones de validación (fondo ámbar).
- Las **reglas de traducción orientadas por la sintaxis – SDT** (texto azul) para **todas** las producciones que generan salida en el lenguaje destino. Deben garantizar que ante cualquier cadena sintáctica y semánticamente válida se produzca una salida válida en el lenguaje destino.

Requisitos de las reglas SDT

- Deben cubrir **todas las producciones** de la gramática que contribuyen a la salida.
- Deben ser consistentes con el lenguaje destino declarado desde el inicio del proyecto.
- Deben manejar correctamente los casos de listas vacías (épsilon) y repeticiones (producción recursiva).
- La cadena de salida resultante debe ser **sintácticamente válida** en el lenguaje destino.

Ejemplo ilustrativo (fragmento de referencia basado en la gramática del curso):

```
<product> → PRODUCT { PRDID { %id% } TITLE { "str" } DESC { "str" } PRICE { num } }
    // REGLA SEM – Declaración de producto
    id.type <- "PRODUCT"   // atributo sintetizado
    SI id ∈ tablaSimbolos → error: "Producto ya declarado"
    SI num <= 0 → error: "El precio debe ser un valor positivo"
    // REGLA SDT – Generación de fila HTML
    { print "<tr><td>" id.title_str "</td><td>" id.desc_str "</td>"
          "<td>" id.price_num "</td></tr>" }
```

Nota: el fragmento anterior es únicamente de referencia. Su gramática debe presentar las SDT propias, coherentes con el lenguaje fuente y destino de su proyecto.

Parte 2 – Compilador completamente funcional

El compilador entregado debe procesar de forma encadenada las siguientes cuatro fases:

#	Fase	Lo que debe hacer el compilador
1	**Análisis léxico**	Tokenizar la cadena de entrada: identificar identificadores, palabras clave, literales de cadena, números y símbolos. Reportar tokens no reconocidos con fila y columna.
2	**Análisis sintáctico**	Verificar que la secuencia de tokens obedece la gramática definida. Reportar errores sintácticos con descripción, fila y columna. Implementar recuperación de errores para continuar el análisis.
3	**Análisis semántico**	Validar las reglas semánticas sobre el árbol/AST: tabla de símbolos, tipos, declaración antes de uso, restricciones de dominio y completitud de campos. Reportar todos los errores semánticos encontrados.
4	**Traducción orientada por la sintaxis (SDT)**	Recorrer el árbol aplicando las acciones SDT definidas en la gramática ampliada para producir el texto de salida en el lenguaje destino. La salida debe ser sintáctica y semánticamente válida en dicho lenguaje.

Requisitos de integración y calidad

- **Flujo encadenado:** el compilador debe ejecutar las cuatro fases en orden. Si se detectan errores semánticos, **no debe generarse código de salida**; en cambio, deben reportarse todos los errores encontrados.
- **Reporte de errores unificado:** todos los errores (léxicos, sintácticos y semánticos) deben reportarse con **fase, fila, columna, lexema y mensaje descriptivo**. El compilador debe continuar el análisis tras cada error para reportar el mayor número posible de problemas en una sola ejecución.

- **Salida del compilador:** cuando la entrada es válida, el compilador produce el texto de salida en el lenguaje destino y lo escribe en un archivo o lo muestra en consola, de forma claramente diferenciada de los mensajes de error.
- **Tabla de símbolos:** debe estar disponible durante la fase SDT para consultar los atributos de los identificadores ya resueltos en el análisis semántico (p. ej. tipo, valor literal, etc.).
- **Separación de módulos:** cada fase debe estar claramente diferenciada en el código (clases, módulos o archivos separados). El evaluador debe poder identificar sin ambigüedad dónde ocurre cada fase.
- **Casos de prueba:** incluir al menos **tres (3) archivos de entrada de prueba**: uno sin errores (que produce salida completa), uno con errores semánticos (sin salida), y uno con errores sintácticos.

Bonus Opcional (+1.0 punto): Validación e Inteligencia Artificial sobre el Código Generado

Una vez que el compilador genera el texto de salida en el lenguaje destino, como actividad opcional puede integrar un modelo de inteligencia artificial que reciba ese código generado y realice sobre él dos tareas: (1) validación sintáctica y semántica del código de salida, y (2) generación de sugerencias de optimización o mejora sobre el mismo.

Modalidades de implementación (elija al menos una):

- Modalidad A – Validación del código generado: el texto producido por el módulo SDT es enviado a un modelo de lenguaje (vía API de Anthropic, OpenAI, Google Gemini u otro) con un prompt que le solicita revisar si el código de salida es sintáctica y semánticamente correcto para el lenguaje destino, reportando cualquier problema encontrado.
- Modalidad B – Sugerencias de optimización: el modelo de lenguaje analiza el código generado y propone mejoras concretas (simplificación de estructura, buenas prácticas del lenguaje destino, accesibilidad, eficiencia, etc.). Las sugerencias se presentan al usuario junto con el código de salida.
- Modalidad C – Combinación de ambas: el compilador primero valida el código generado con IA y luego, si es válido, solicita al mismo modelo sugerencias de optimización. El resultado final muestra al usuario el código generado, el reporte de validación y las sugerencias en un único informe.

Requisitos para obtener el punto bonus:

- El documento debe describir la modalidad elegida y el prompt utilizado para interactuar con el modelo de IA.
- El código fuente debe mostrar claramente la integración: llamada a la API, manejo de la respuesta y presentación al usuario.
- Presentar al menos un ejemplo completo: código fuente de entrada → código generado por el compilador → respuesta de la IA (validación y/o sugerencias).
- La integración con IA es una capa adicional: el compilador debe funcionar correctamente de forma autónoma aunque la IA no esté disponible.

Nota: se valora la utilidad práctica de las sugerencias generadas, la calidad del prompt utilizado y la claridad con que se presenta el resultado al usuario final.

Entregables

Se deben cargar en Moodle los siguientes archivos antes del cierre del plazo:

- **Documento PDF** con la gramática ampliada final, usando la convención de colores establecida: reglas originales en rojo, reglas semánticas en fondo ámbar y reglas de traducción SDT en azul.
- **Código fuente completo del compilador** (comprimido en .zip), incluyendo: todos los módulos del compilador (léxico, sintáctico, semántico, SDT), los tres archivos de prueba requeridos, y un archivo README con instrucciones claras para compilar y ejecutar el proyecto.
- Presentación del proyecto de máximo 10 minutos

- (Opcional – Bonus) Evidencia de la integración con IA: descripción del prompt utilizado, fragmento de código de la integración y al menos un ejemplo completo (entrada → salida del compilador → respuesta de la IA).

Rúbrica de evaluación

Criterio de calificación	Puntos	Observaciones
El documento PDF presenta la gramática ampliada completa con las reglas semánticas (≥ 5) y todas las reglas de traducción orientadas por la sintaxis (SDT), usando la convención de colores establecida (rojo: gramática base, fondo ámbar: semántica, azul: SDT). Cada regla semántica indica sus atributos, dominio de valores y acciones de validación. Cada regla SDT cubre la producción correspondiente y garantiza una salida válida en el lenguaje destino.	1.5	Rojo: gramática base. Ámbar: semántica. Azul: SDT.
El compilador entregado integra correctamente las cuatro fases (léxico → sintáctico → semántico → SDT): identifica y reporta errores léxicos, sintácticos y semánticos con fila, columna, lexema y mensaje descriptivo; implementa recuperación de errores; no genera salida si hay errores semánticos; y produce un texto válido en el lenguaje destino para entradas correctas.	2.0	Se evaluarán los tres archivos de prueba obligatorios incluidos en el entregable.
La presentación realizada expone la funcionalidad básica del proyecto, en ella intervienen todos los integrantes del equipo sin la ayuda de notas, celulares y dirigiéndose a la audiencia	1.5	
Bonus opcional – Validación e IA sobre el código generado (ver sección correspondiente).	**+1.0**	Opcional. No afecta la nota base.
Total (sin bonus)	**5.0**	